Efficient Key-Aggregate Cryptosystem With User Revocation for Selective Group Data Sharing in Cloud Storage

Jinlu Liu, Jing Qin, Xi Zhang, and Huaxiong Wang

***Abstract*—Cloud computing has become prevalent due to its extensive storage resources and robust computational capacities. To protect data security and privacy, data owners opt for uploading encrypted data to the cloud. Flexible sharing of these encrypted data in a group of users is a critical functionality in cloud storage. In addition, given that users may exit the group, revocation becomes a crucial requirement in group data-sharing systems. The Key-Aggregate Cryptosystem (KAC) has become a promising mechanism for group data sharing. The decryption rights for any set of ciphertexts can be efficiently delegated by distributing a constant-size aggregate key, while the confidentiality of other ciphertexts outside the set is maintained. However, in previous KAC schemes, revocation remains a challenging task regarding key update, ciphertext re-encryption, and collision resistance. In this paper, we propose a Key-Aggregate Cryptosystem with User Revocation (KAC-UR) scheme to overcome this challenge. The KAC-UR scheme not only achieves flexible data sharing, but also can perform secure and efficient user revocation with properties including collision resistance, revocation without data owner-user communication, and constant ciphertext size. The KAC-UR scheme also enables the cloud server to perform partial decryption, thereby significantly alleviating the computational burden for users. The KAC-UR scheme is chosen plaintext attack secure under the decisional Bilinear Diffie-Hellman Exponent assumption.**

***Index Terms*—Cloud computing, data sharing, key-aggregate cryptosystem, revocation.**

I. Introduction

With the fast advancement of cloud computing [1], [2], cloud storage service is becoming more and more mature, and it has received extensive attention from academia and industry because of its scalability, accessibility, and efficient backup of data. However, in the cloud storage service mode, how to protect data security becomes a potential security issue because the data is out of users' control [3], [4], [5]. A direct way to protect data privacy is encryption before outsourcing [6], [7], [8], [9]. Nevertheless, encryption limits the flexibility of access control when the data is being shared with a group of users. In particular, data owners outsource much of their data to the cloud server but solely want to share part of them, other data remain private.

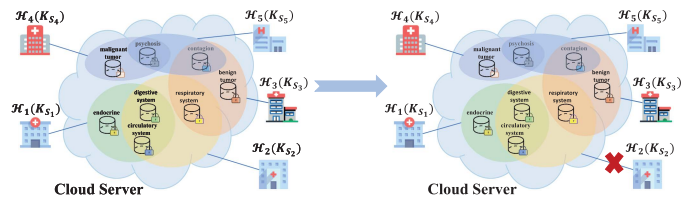

Fig. 1. Collaboration data sharing scenario.

The key-aggregate cryptosystem (KAC) [10] is an excellent technology that enables data owners to securely, efficiently, and selectively share data in cloud storage with a group of users. In KAC, all data are divided into different classes, and the data owner encrypts data under the master public key and an identifier that indicates which class the data belongs to. The resulting ciphertext is constant size. Further, the data owner can extract an aggregate key from any number of different classes using his master secret key. Although this aggregate key is a single key, it can decrypt ciphertexts of these different classes. This means that when the data owner shares a subset S of the data classes with the user, it simply sends the corresponding constant size aggregate key K_S to the user via the secure channel.

In addition, data sharing is not static. When users' authorization is expired, their access permissions should be revoked. For instance, a departing employee no longer has access to the internal information of the company; in a collaborative data sharing scenario, the data of the partner cannot be accessed after the collaboration terminates. To further illustrate this problem, consider the following realistic collaboration data sharing scenario.

As shown in Fig. 1, Hospital $\mathcal{H}$ divides all its patients' data into the following classes by disease type: contagion, malignant tumor, benign tumor, psychosis, respiratory system, circulatory system, digestive system, and endocrine. To protect patient

Manuscript received 15 September 2023; revised 22 April 2024; accepted 3 May 2024. Date of publication 7 May 2024; date of current version 27 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62072276 and Grant 61772311, and in part by the Singapore Ministry of Education Academic Research Fund Tier 2 under Grant MOE2019-T2-2-083. Recommended for acceptance by X. Yi. *(Corresponding author: Jing Qin.)*

Jinlu Liu is with the School of Mathematics, Shandong University, Jinan, Shandong 250100, China (e-mail: jinlulmath@163.com).

Jing Qin is with the School of Mathematics, Shandong University, Jinan, Shandong 250100, China, and also with the Skate Key Laboratory of Cryptology, Beijing 100878, China (e-mail: qinjing@sdu.edu.cn).

Xi Zhang is with the School of Mathematics, Shandong University, Jinan, Shandong 250100, China (e-mail: mathxizhang@mail.sdu.edu.cn).

Huaxiong Wang is with the School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore 639798 (e-mail: hxwang@ntu.edu.sg).

Digital Object Identifier 10.1109/TKDE.2024.3397721

1041-4347 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

privacy and selectively share data with other hospitals, $\mathcal{H}$ encrypts these data using KAC and then uploads them to cloud storage. We assume that in order to cooperate, $\mathcal{H}$ needs to share data subset S_1={circulatory system, digestive system, endocrine } to Hospital $\mathcal{H}_1$, S_2={ respiratory system, circulatory system, digestive system } to Hospital $\mathcal{H}_2$, S_3={ contagion, malignant tumor, benign tumor, respiratory system } to Hospital $\mathcal{H}_3$, S_4={ malignant tumor, psychosis } to Hospital $\mathcal{H}_4$, and S_5={ contagion, psychosis} to Hospital $\mathcal{H}_5$. $\mathcal{H}$ only extracts and sends aggregate key K_{S_1}, K_{S_2}, K_{S_3}, K_{S_4}, and K_{S_5} to hospitals $\mathcal{H}_1$, $\mathcal{H}_2$, $\mathcal{H}_3$, $\mathcal{H}_4$, and $\mathcal{H}_5$ respectively. At some point, the collaboration between $\mathcal{H}$ and $\mathcal{H}_2$ ends, and $\mathcal{H}_2$ will no longer have access to data in S_2, implying that $\mathcal{H}_2$'s access to S_2 should be revoked.

However, the corresponding aggregate key K_{S_2} is possessed by $\mathcal{H}_2$ when $\mathcal{H}$ authorizes it to access S_2 and remains unchanged subsequently. This becomes troublesome when there's a need to revoke access permissions. With the existing solutions [11], [12], when the revocation needs to be performed in KAC, one way is to update the key and re-encrypt the ciphertext [11]. This method is trivial. Specifically, a dynamic access control parameter is introduced so that the ciphertext can only be decrypted if the user has both the aggregate key and the access control parameter. When there are users that need to be revoked, the data owner first updates the access control parameter. Then, using the new access control parameter, he re-encrypts and re-uploads data that was encrypted before these users revoked. Finally, the data owner securely transmits the new parameter to unrevoked users. This method, however, has the following limitations: 1) At each revocation, each unrevoked user must be online and communicate with the data owner over the secure channel to get the new access control parameter and store it securely. Hence, the overhead of secure communication is considerable. 2) At each revocation, the re-encryption of the ciphertext requires the data owner to go through the process of "*download*"-"*decrypt*"-"*re-encrypt*"-"*re-upload*". This process brings additional significant computation, communication, and data maintenance overheads to the data owner. 3) Each user's access control parameter is the same. Consequently, a revoked user can still use the aggregate key he previously obtained, alongside the updated access control parameter acquired from an unrevoked user, to successfully decrypt the ciphertext. As a result, this method cannot resist collusion attacks between the revoked user and the unrevoked user. Another way is to update the ciphertext [12]. Specifically, this method is based on the multilinear map [13] and subset-cover framework [14]. When the need arises to revoke users, the cloud server updates all ciphertexts according to the user revocation set R, rendering the revoked users unable to decrypt the updated ciphertexts. However, using this method, the ciphertext size is related to the number of revoked users, rather than remaining constant. In addition, the scheme relies on the multilinear map. In terms of security, the multilinear map is an unstable assumption. In terms of efficiency, the multilinear map demands very expensive computations. These limitations restrict these schemes from being utilized in practice.

A. Our Contributions

In this paper, our motivation is to tackle the above-mentioned limitations and propose a secure and practical data sharing scheme in cloud environments. This scheme aims to offer selective data sharing and enable the revocation of user access rights. To achieve these objectives, we make the following contributions:

- We propose a novel framework known as the Key-Aggregate Cryptosystem with User Revocation (KAC-UR). This framework allows data owners to securely store data in the cloud by applying KAC, thus enabling them to selectively share encrypted data effectively. More importantly, our KAC-UR framework offers an efficient revocation mechanism for data owners, accomplished through key updating. The data are encrypted and then outsourced to the cloud based on two attributes: data class and time period. To decrypt a ciphertext encrypted at time period $\mathcal{T}_j$, a user u not only needs the corresponding aggregate key but also needs an updated key corresponding to u and $\mathcal{T}_j$. Consequently, the data owner can revoke a user's access to data by stopping the generation of updated keys for that user.
- We instantiate a KAC-UR scheme by leveraging Chu et al.'s KAC scheme [10] and Shamir secret sharing [15]. In the KAC-UR scheme, the data owner encrypts his data for two attributes: data class and time period. By utilizing the Shamir secret sharing scheme, the data owner splits the original aggregate key into two components: aggregate key and updated key. Our proposed scheme supports flexible data sharing and user revocation with the following features: 1) *Collision resistance:* the aggregate keys and updated keys of different users can't be combined in any meaningful way to decrypt a ciphertext that they cannot decrypt alone; 2) *Revocation without data owner-user communication:* due to collision resistance, the updated keys can be publicly stored in the cloud server, eliminating the need for users to communicate with the data owner via secure channels. At each time period, the server stores the newly updated keys of the unrevoked users while deleting the updated keys of all time periods of the revoked user to prevent the revoked users from decrypting ciphertext encrypted before being revoked; 3) *Constant ciphertext size:* our revocation mechanism doesn't need to update ciphertext, so the size of ciphertext keeps constant as in the KAC system; 4) *Outsourced decryption:* the server can use the updated key to perform partial decryption operations for the user, thereby effectively reducing the user's computational burden.
- We define a security model for the proposed KAC-UR framework. The security model captures that data belonging to data class l and encrypted at time period $\mathcal{T}_j$ can only be accessed by a user who is authorized to access a data class set S satisfying $l \in S$ and has not been revoked at the time period $\mathcal{T}_j$. We prove the security of the instantiated KAC-UR scheme under the decisional Bilinear Diffie-Hellman Exponent (BDHE) assumption. We also

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FUNCTIONALITY COMPARISON WITH [11] AND [12]

Schemes	Revocation Mode	Collusion Resistant	Size of Ciphertext is Constant	No Secure Channel is Required at Revocation	Outsourced Decryption	Undering Group
Patranabis et al. [11]	Key Updating Ciphertext Re-encryption	×	✓	×	×	Bilinear Group
Gan et al. [12]	Ciphertext Updating	✓	×	✓	×	Multilinear Group
Our Scheme	Key Updating	✓	✓	✓	✓	Bilinear Group

conduct theoretical and practical comparative analysis of the KAC-UR scheme with related schemes. The results show the KAC-UR scheme's high efficiency.

To show the superiorities of our scheme, in Table I, we compare our scheme in detail with existing representative revocable KACs in terms of functionality. Section V will deal with the efficiency comparison.

B. Related Works

Data sharing [16], [17], [18], [19], [20] is an important functionality of cloud storage that has received extensive attention recently. To solve the problem of key management in flexible data sharing, Chu et al. [10] introduced the key-aggregate cryptosystem (KAC). It allows a single aggregate key to decrypt multiple data classes encrypted using different keys. [10] also proposed a concrete KAC scheme using the broadcast encryption scheme [21], but the cryptographic security model and security proof were not provided. Since then, many KAC systems have been proposed aiming at its provable security and needs for different application scenarios.

Patranabis et al. [22] proposed chosen plaintext attack (CPA) and chosen ciphertext attack (CCA) secure KAC schemes by using asymmetric bilinear pairing. They also used broadcast encryption to solve the issue of distributing aggregate keys to multiple users. Wang et al. [23] designed two leakage-resilient KAC schemes by modifying Patranabis et al.'s scheme [22]. Guo et al. [24] pointed out that the KAC scheme of [10] has two limitations. One is that it does not support dynamic cloud storage, that is, the maximum number n of data classes is fixed. When the data class increases to more than n, new system parameters need to be generated. The other is that the key leakage problem is not considered. Therefore, Guo et al. [24] designed an authentication KAC system in which n is variable, and the authentication process solves the key-leakage problem. However, Alimohammadi et al. [25] investigated the vulnerability of Guo et al.'s scheme to DoS and impersonation attacks and found out that anyone could forge the authentication key. To overcome this weakness, Alimohammadi et al. [25] designed a new scheme for dynamic cloud storage. In 2024, Liu et al. [26] investigated the data leakage problem of KAC in the presence of malicious data owners. They proposed two key-aggregate based access control encryption (KA-ACE) schemes that can provide flexible bidirectional access control for senders and receivers.

Patranabis et al. [11] first attempted to solve the problem of user access revocation in KAC. Their approach introduced a dynamic access control parameter, ensuring that decryption of a ciphertext is only possible when a user possesses both the aggregate key and the corresponding access control parameter. When some users are revoked, the data owner must distribute a new access control parameter to each unrevoked user over the secure channel, which requires substantial secure communication overhead. In addition, [11] can not resist collusion attacks. Gan et al. [12] designed a revocable KAC scheme based on the multilinear map [13] and subset-cover framework [14]. When some users are revoked, the cloud server can update ciphertexts so that revoked users cannot access the new ciphertexts. However, after revocation, ciphertext size is no longer a constant but depends on the number of users being revoked. Moreover, the multilinear map is an unstable assumption and requires very expensive computations.

KAC is also combined with other cryptographic primitives, for instance, proxy re-encryption [27], identity-based encryption [28], time-bound encryption [29], and searchable encryption [30], [31], [32], [33].

C. Organization

The rest of this paper is structured as follows. In Section II, we give the relevant background knowledge. In Section III, we give the system architecture, system algorithm, and security model. The concrete construction and security proof are proposed in Section IV. Section V gives the performance analysis. Finally, we conclude this whole paper in Section VI.

II. PRELIMINARIES

A. Bilinear Pairing

Let $\mathcal{G}$ and $\mathcal{G}_1$ be two cyclic multiplicative groups of prime order p, and g be a generator of $\mathcal{G}$. A map $e : \mathcal{G} \times \mathcal{G} \to \mathcal{G}_1$ is a bilinear pairing if it satisfies the following properties:

- Bilinearity: For any $u, v \in \mathcal{G}$, $x, y \in Z_p$, there is $e(u^x, v^y) = e(u, v)^{xy}$.
- Non-degeneracy: $e(g, g) \neq 1$.
- Computability: For any $u, v \in \mathcal{G}$, there is an efficient algorithm to calculate $e(u, v)$.

B. Bilinear Diffie-Hellman Exponent (BDHE) Assumption

n-BDHE Problem: Given a vector of $2n + 1$ elements

$$(h, g, g^{\alpha}, g^{(\alpha^2)}, \ldots, g^{(\alpha^n)}, g^{(\alpha^{n+2})}, \ldots, g^{(\alpha^{2n})}) \in \mathcal{G}^{2n+1}$$

as input, output $e(g, h)^{(\alpha^{n+1})}$. For shorthand, when g and α are given, we use g_l to denote $g^{(\alpha^l)}$.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

The advantage of an algorithm $\mathcal{A}$ to solve n-BDHE problem is ε if

$$Pr[\mathcal{A}(h, g, g_1, g_2, \ldots, g_n, g_{n+2}, \ldots, g_{2n}) = e(g_{n+1}, h)] \geq \varepsilon.$$

Decisional n-BDHE Problem: Given a vector of $2n+1$ elements

$$(h, g, g_1, g_2, , \ldots, g_n, g_{n+2}, \ldots, g_{2n}) \in \mathcal{G}^{2n+1}$$

and $Z \in \mathcal{G}_1$, the decisional version of n-BDHE problem is to decide $Z \stackrel{?}{=} e(g_{n+1}, h)$. For shorthand, let $y_{g,\alpha,n} = (g_1, g_2, \ldots, g_n, g_{n+2}, \ldots, g_{2n})$.

The advantage of an algorithm $\mathcal{A}$ with output $b \in \{0, 1\}$ to solve decisional n-BDHE problem is ε if

$$|Pr[\mathcal{A}(g, h, y_{g,\alpha,n}, e(g_{n+1}, h)) = 1] - Pr[\mathcal{A}(g, h, y_{g,\alpha,n}, Z) = 1]| \geq \varepsilon.$$

We refer to the distribution of $\{g, h, y_{g,\alpha,n}, e(g_{n+1}, h)\}$ as T_{BDHE} and the distribution of $\{g, h, y_{g,\alpha,n}, Z\}$ as F_{BDHE}.

Definition 1 (BDHE Assumption): We say that the (decisional) n-BDHE assumption holds in $\mathcal{G}$ if the advantage of solving the (decisional) n-BDHE problem is negligible (in security parameter λ) for any probabilistic polynomial time (PPT) algorithm $\mathcal{A}$.

C. Shamir's (t,n) Threshold Secret Sharing Scheme

Definition 2: Let t, n be positive integers, and a (t, n) threshold scheme refers to such a way that n participants (the set of participants is denoted as $\mathcal{P}$) share a secret K: any t participants can calculate the value of K, but any $t-1$ participants cannot calculate the value of K.

In 1979, Shamir [15] proposed a (t, n) threshold secret sharing scheme based on Lagrange interpolation formula. This scheme consists of the following algorithms:

- $Setup$: Secret distributor D randomly selects n different nonzero elements $\{x_1, x_2, \ldots, x_n\}$ from Z_p (p is prime and $p \geq n+1$). D sends x_i to P_i ($P_i \in \mathcal{P}$), and x_i is public.
- $Divide\text{-}Secret$: If D wants to share a secret $K \in Z_p$, it performs the following:
  a) D secretly selects (independently randomly selects) $t-1$ elements in Z_p, denoted as $a_1, a_2, \ldots, a_{t-1}$, respectively.
  b) For $1 \leq i \leq n$, D computes $y_i = a(x_i)$, where
  $$a(x) = K + \sum_{j=1}^{t-1} a_j x^j \bmod p.$$
  c) For $1 \leq i \leq n$, D secretly sends y_i to P_i as his sub-secret.
- $Recover\text{-}Secret$: Any t of n participants may be set to $P_1, P_2, .., P_t$ to show their sub-secrets and thus get t point pairs: (x_1, y_1), $(x_2, y_2), \ldots, (x_t, y_t)$, so that the polynomial $a(x)$ and the shared secret value K can be reconstructed as follows:
  $$a(x) = \sum_{i=1}^{t} y_i \left(\prod_{j=1, j\neq i}^{t} \frac{x - x_j}{x_i - x_j} \right) \bmod p,$$
  $$K = a(0) = \sum_{i=1}^{t} y_i \left(\prod_{j=1, j\neq i}^{t} \frac{x_j}{x_j - x_i} \right) \bmod p.$$

D. The Brief Description of Chu et al.'s KAC Scheme

The Chu et al.'s KAC scheme (Chu-KAC) [10] is given as follows.

- $Setup(1^\lambda, n)$: Given the security parameter λ and the maximum number of data classes n (the class identifier should be an integer bounded by 1 and n), the data owner initializes system as follows:
  a) Generate a bilinear map group system $\mathcal{PG} = (p, \mathcal{G}, \mathcal{G}_1, e(\cdot, \cdot))$, where p is the order of $\mathcal{G}$.
  b) Pick a random generator $g \in \mathcal{G}$ and a random $\alpha \in Z_p$, and compute $g_l = g^{(\alpha^l)} \in \mathcal{G}$ for $l = 1, 2, \ldots, n, n+2, .., 2n$.

  Return $param = \langle \mathcal{PG}, g, g_1, g_2, .., g_n, g_{n+2}, \ldots, g_{2n} \rangle$.
- $KeyGen()$: The data owner randomly picks $w \in Z_p$ as his master secret key (MSK), and computes $W = g^w$ as master public key (MPK).
- $Encrypt(MPK, l, M)$: For a data M belonging to class l, the data owner randomly selects $z \in Z_p$ and computes the ciphertext as $CT_l = (C_0, C_1, C_2)$, where
  $$C_0 = M \cdot e(g_1, g_n)^z, C_1 = g^z, C_2 = (W \cdot g_l)^z.$$
- $Extract(MSK, S)$: The data owner computes the aggregate key for a subset $S \subseteq [1, n]$ to be $K_S = \prod_{k\in S} (g_{n+1-k})^w$ and securely sends $\{K_S, S\}$ to the user.
- $Decrypt(K_S, CT_l)$: For a ciphertext CT_l where $l \in S$, the user returns the data as
  $$M = C_0 \cdot \frac{e(K_S \cdot \prod_{k\in S, k\neq l} g_{n+1-k+l}, C_1)}{e(\prod_{k\in S} g_{n+1-k}, C_2)}.$$

III. Problem Statement

In this section, we present the system architecture, system algorithms, threat model, and security model of our scheme. Before going into the detailed description, we illustrate the main notations in Table II.

A. System Architecture

Our KAC-UR system consists of three types of entities: data owner (DO), data user (DU), and cloud server (CS). We describe how our system works in Fig. 2.

- *(Encryption: data owner → cloud server)* The data owner encrypts the data under his public key, current time period, and data class identifier. Then, he uploads the generated ciphertext to the server.
- *(Aggregate Key Distribution: data owner → data user)* The data owner calculates an aggregate key of multiple data classes using his master secret key and provides it to the user over a secure channel.
- *(Key Updating: data owner → cloud server)* The data owner transmits a set of updated keys to the server during key updating.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATIONS

Notation	Description
PPT	Probability polynomial time
λ	Security parameter
n	The maximum number of data classes owned by a data owner
$\mathcal{T}_j$	The jth time period
(MPK, MSK)	The master public key and master secret key pair of the data owner
(PK_j, SK_j)	The public key and secret key pair of the data owner at time period $\mathcal{T}_j$
RI_j	The identities of the users revoked at time period $\mathcal{T}_j$
RL_j	The revoction list at time period $\mathcal{T}_j$
u_i	The identity of ith data user
S_i	The identifier set of data classes that authorize u_i to access
K_{S_i,u_i}	The aggregate key sent to the u_i
$ku^j_{S_i,u_i}$	The updated key for u_i at time period $\mathcal{T}_j$
CT^j_l	A ciphertext belonging to class l encrypted at time period $\mathcal{T}_j$
CT^j_{l,u_i}	The partially decrypted ciphertext obtained by decrypting CT^j_l using $ku^j_{S_i,u_i}$

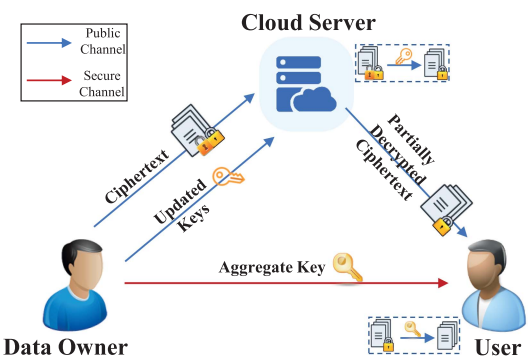

Fig. 2. The KAC-UR system architecture.

- *(Partial Decryption: cloud server → data user)* The server converts the ciphertext into a partially decrypted ciphertext using an updated key corresponding to the user and the time period nested in the ciphertext.
- *(Decryption: data user)* The user recovers the original data of the data owner from the partially decrypted ciphertext using the received aggregate key.

B. System Algorithms

Definition 3 (KAC-UR): A KAC-UR system consists of the following algorithms:

- $Sys(1^\lambda, n) \rightarrow param$: The system parameter generation algorithm is run by the data owner. On input a security parameter λ and the maximum number of data classes n, it outputs the system parameter $param$.
- $Setup(\mathcal{T}_0) \rightarrow (MPK, MSK, PK_0, SK_0, RL_0)$: The setup algorithm is run by the data owner. It outputs the data owner's master public key and master secret key pair (MPK, MSK), the public key and secret key pair (PK_0, SK_0) of initial time $\mathcal{T}_0$, and the initial revocation list RL_0.
- $Extract(MSK, u_i, S_i) \rightarrow K_{S_i,u_i}$: The aggregate key extraction algorithm is run by the data owner. On input MSK, a user's identity u_i, and an identifier set $S_i \subseteq [1, n]$ of all data classes that authorize u_i to access, it outputs the aggregate key K_{S_i,u_i}. The data owner transmits $\{K_{S_i,u_i}, S_i\}$ to the user via secure channel.
- $Update(\mathcal{T}_j) \rightarrow (PK_j, SK_j)$: The update algorithm is run by the data owner. On input the current time period $\mathcal{T}_j$, it updates public key and secret key pair (PK_{j-1}, SK_{j-1}) of last time period $\mathcal{T}_{j-1}$ to (PK_j, SK_j).
- $Enc(MPK, PK_j, l, M) \rightarrow CT^j_l$: The encryption algorithm is run by the data owner. On input MPK, PK_j, an identifier l of data class, and a data M belonging to class l, it outputs a ciphertext CT^j_l.
- $Revoke(RL_{j-1}, \mathcal{T}_j, RI_j) \rightarrow RL_j$: The revocation algorithm is run by the data owner and the cloud server. On input the revocation list RL_{j-1} at time period $\mathcal{T}_{j-1}$, the current time period $\mathcal{T}_j$, and the identity set RI_j of users revoked at this time period, it outputs an updated revocation list RL_j. The data owner also sends the RI_j to the cloud server. Then, for each $u_i \in RI_j$, the cloud server delete all his updated keys set $\{ku^1_{S_i,u_i}, \ldots, ku^{j-1}_{S_i,u_i}\}$.
- $KeyUp(MSK, SK_j, RL_j, u_i, S_i) \rightarrow ku^j_{S_i,u_i}$: The key update generation algorithm is run by the data owner. On input MSK, SK_j, RL_j, a user identity $u_i \notin RL_j$, and an identifier set $S_i \subseteq [1, n]$ of all data classes that authorize u_i to access, it outputs an updated key $ku^j_{S_i,u_i}$ for u_i at time period $\mathcal{T}_j$. We record the set of user-updated key pairs of all unrevoked users at time period $\mathcal{T}_j$ as KU_j, namely $KU_j = \{u_i, ku^j_{S_i,u_i}\}_{u_i \notin RL_j}$. The data owner transmits $\{\mathcal{T}_j, KU_j\}$ to the server (via a public channel).
- $Transform(ku^j_{S_i,u_i}, CT^j_l) \rightarrow CT^j_{l,u_i}$: The ciphertext transformation algorithm is run by the cloud server. On input the updated key $ku^j_{S_i,u_i}$ for user u_i at time period $\mathcal{T}_j$ and a ciphertext CT^j_l belonging to class $l \in S_i$ uploaded at time period $\mathcal{T}_j$, it outputs a partially decrypted ciphertext CT^j_{l,u_i}. The server sends CT^j_{l,u_i} to user u_i (via a public channel).
- $Dec(K_{S_i,u_i}, CT^j_{l,u_i}) \rightarrow M$: The decryption algorithm is run by the user u_i. On input the aggregate key K_{S_i,u_i} and the partially decrypted ciphertext CT^j_{l,u_i}, it outputs a data M.

Definition 4 (Correctness of KAC-UR): For all security parameter λ and all data M, if every entity honestly executes the algorithms, and if the user u_i is not revoked until time period $\mathcal{T}_j$ and $l \in S_i$, then we have $Dec(K_{S_i,u_i}, Transform(ku^j_{S_i,u_i}, CT^j_l)) = M$.

C. Threat Model and Security Model

We assume that the cloud server is a semi-trusted entity like many previous schemes. That is, it honestly conducts the protocol, but tries to spy out as much secret information as possible by

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

analyzing the statistics of outsourced data, resulting in a serious security breach in the confidentiality of the encrypted raw data. For example, one company uses cloud servers such as Google Drive, Dropbox, and SugarSync as its file sharing platform, where employees can upload sensitive files such as customer reports, sales data, and financial information to the cloud server and share them with other team members or partners. In this case, the cloud server may try to learn about the company's business or customer information by analyzing the contents of the uploaded file. The data owner is trustworthy. Unauthorized users (those having no correct aggregate key or updated key) may attempt to expose the data owner's ciphertexts. For example, one company stores its sales reports on a cloud server and shares them with specific team members. However, some unauthorized users (e.g., non-specific team members, people outside the company, competitors, hackers, etc.) may attempt to obtain information from these sales reports by pretending to be authorized users in order to gain access to competitors' trade secrets or other sensitive information.

Considering the above-mentioned realistic attacks, we propose the formal security definition of indistinguishability under chosen plaintext attacks (IND-CPA). This attack is launched by unauthorized users and semi-trusted CS. Security is defined by the following IND-CPA game between attack algorithm $\mathcal{A}$ and challenger $\mathcal{C}$. Both $\mathcal{C}$ and $\mathcal{A}$ are given n as input.

IND-CPA Game

- *Init:* $\mathcal{A}$ declares the data class identifier $l^* \in [1, n]$ and the time period $\mathcal{T}_*$ that it wishes to attack.
- *Sys:* $\mathcal{C}$ runs the $Sys(1^\lambda, n)$ algorithm and sends the resulting $param$ to $\mathcal{A}$.
- *Setup:* $\mathcal{C}$ runs the $Setup(\mathcal{T}_0)$ algorithm and provides the resulting master public key MPK, public key PK_0, and initial revocation list RL_0 to $\mathcal{A}$ while master secret key MSK and secret key SK_0 are kept secret from $\mathcal{A}$.
- *Query Phase 1:* $\mathcal{A}$ adaptively issues the following queries polynomial times.
  - Aggregate key query, which consists of a tuple (u_i, S_i) with $S_i \subseteq [1, n]$. $\mathcal{C}$ runs $Extract(MSK, u_i, S_i)$ and gives the resulting aggregate key K_{S_i,u_i} to $\mathcal{A}$.
  - Update query $(\mathcal{T}_j)$. $\mathcal{C}$ runs $Update(\mathcal{T}_j)$ to generate public key and secret key pair (PK_j, SK_j) and sends PK_j to $\mathcal{A}$.
  - Revocation query, which consists of a tuple $(\mathcal{T}_j, RI_j)$. $\mathcal{C}$ runs $Revoke(RL_{j-1}, \mathcal{T}_j, RI_j)$ and sends the resulting updated revocation list RL_j to $\mathcal{A}$. The restriction is that if $\mathcal{A}$ requests an aggregate key for (u_i, S_i) satisfying $l^* \in S_i$, then the revocation query must be queried on $(\mathcal{T}_j, RI_j)$ with $u_i \in RI_j$ for any $\mathcal{T}_j \leq \mathcal{T}_*$.
  - Key update query, which consists of a tuple $(u_i, S_i, \mathcal{T}_j)$ with $u_i \notin RL_j$. $\mathcal{C}$ runs $KeyUp(MSK, SK_j, RL_j, u_i, S_i)$ and sends the resulting updated key $ku^j_{S_i,u_i}$ to $\mathcal{A}$.
- *Challenge:* When $\mathcal{A}$ determines the end of Query Phase 1, it submits two messages of the same length, M_0 and M_1, belonging to class l^*. $\mathcal{C}$ selects a random bit b, runs the $Enc(MPK, PK_*, l^*, M_b)$, and returns the resulting challenge ciphertext CT^*.
- *Query Phase 2:* $\mathcal{A}$ continues the querying as in Query Phase 1.
- *Guess:* Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ for b.

We define the advantage of $\mathcal{A}$ winning the game as

$$Adv^{IND-CPA}_{KAC-UR}(\lambda) = |Pr[b' = b] - \frac{1}{2}|.$$

Definition 5 (IND-CPA Security): We say that a KAC-UR system is IND-CPA secure if for any PPT adversary $\mathcal{A}$, the $Adv^{IND-CPA}_{KAC-UR}(\lambda)$ is negligible in security parameter λ.

The game above models an attack where users (including the CS) who are not authorized to access data class l^* and users who are authorized to access data class l^* but revoked before or at time period $\mathcal{T}_*$ collude to try and expose a data belonging to class l^* and uploaded at time period $\mathcal{T}_*$. Noting that the adversary is non-adaptive, it chooses l^* and $\mathcal{T}_*$ before it sees $param$, MPK, PK_0, and RL_0.

IV. Proposed KAC-UR Scheme

A. Technical Overview

1) A Trivial Approach: One trivial approach for realizing the revocation of the key aggregate cryptosystem is to embed the time period in the ciphertext and aggregate key, as in identity-based encryption (IBE) [34]. The data owner regularly updates aggregate keys for authenticated users. When a user should be revoked at a certain time period, the data owner simply stops distributing the updated aggregate key for the user.

However, in practice, this trivial method has the following limitations: all unrevoked users must regularly communicate with the data owner through secure channels to obtain their new aggregate keys; The unrevoked user must securely store the aggregate key for each time period. With the update of the time period, this will become a bottleneck. In addition, the revoked user can still decrypt the ciphertexts encrypted before being revoked using previously obtained aggregate keys.

2) Our Efficient Approach: Using Shamir secret sharing, we propose an efficient KAC-UR scheme, to overcome the above limitations. Concretely, in KAC-UR, the data is encrypted for two attributes: data class and time period (In previous schemes, only the data class is associated). Correspondingly, we split the aggregate key into two components. As shown in Fig. 3, when the data owner creates an aggregate key for a user u_i, he associates a random first-order polynomial $q_i(x)$ with u_i and limits the value of polynomial q_i at point 0 to the master secret key, that is, $q_i(0) = w$. Then, we use Shamir's method of secret sharing to distribute shares of the master secret key in the exponents of the user's aggregate key components, so that the original aggregate key $K_{S_i} = \prod_{k \in S_i}(g_{n+1-k})^w$ is divided into $K_{S_i,u_i} = \prod_{k \in S_i}(g_{n+1-k})^{\frac{q_i(1)}{y}}$ and $ku^j_{S_i,u_i} = \prod_{k \in S_i}(g_{n+1-k})^{\frac{q_i(x_i)}{t_j}}$, which are called aggregate key and updated key respectively. The K_{S_i,u_i} indicates that u_i has access to set S_i at the beginning, and $ku^j_{S_i,u_i}$ indicates that u_i's access to set S_i has not been revoked at time period $\mathcal{T}_j$. In order to decrypt the ciphertext in class l outsourced at time period $\mathcal{T}_j$, u_i needs both aggregate key K_{S_i,u_i} (where $l \in S_i$) and updated

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

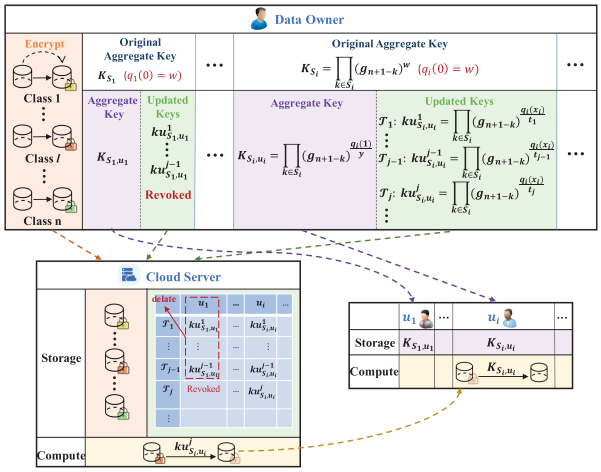

Fig. 3. The main idea of the proposed KAC-UR.

key $ku^j_{S_i,u_i}$. Thus, if the data owner wants to revoke a user, he simply stops publishing the updated key for that user.

So far, this approach seems to have the same limitations as the trivial one (unrevoked users must receive their updated keys through secure channels and securely store these updated keys, and the revoked user can still decrypt the ciphertexts encrypted before being revoked). But notice that, as with fuzzy identity-based encryption [35], aggregate keys for different users are generated with different random polynomials, so the aggregate keys and updated keys of different users cannot be combined in any meaningful way to decrypt a ciphertext that they cannot decrypt alone. Based on this observation, the data owner can solely send the aggregate key to the user, but the updated key, which needs to be updated periodically, can be stored on the cloud server. This not only makes each user securely store just a constant-size aggregate key but also eliminates the need for communication between the user and the data owner during key updating. To prevent the revoked user from decrypting ciphertexts encrypted before being revoked, at each time period, the data owner sends the identity set of revoked users to the server, which deletes the updated keys of these users for all time periods. In addition, the server can perform the ciphertext transformation algorithm using the updated key to convert a ciphertext into a partially decrypted ciphertext and then send it to the user, significantly reducing the user's decryption overhead.

Therefore, with the KAC-UR scheme, the data owner can selectively share encrypted data with users efficiently like the KAC mechanism. What's more, the data owner can securely and efficiently revoke the access right of a user at any time period. At last, outsourced decryption effectively reduces the user's computational overhead.

B. Concrete Construction

Let $\triangle_{i,C}$ denote the Lagrange cofficient for $i \in Z_p$ and a set C, namely

$$\triangle_{i,C}(x) = \prod_{j \in C, j \neq i} \frac{x-j}{i-j}.$$

Our proposed KAC-UR scheme contains the following algorithms.

- $Sys(1^\lambda, n)$: Same as Chu-KAC's $Setup(1^\lambda, n)$ algorithm.
- $Setup(\mathcal{T}_0)$: On input $\mathcal{T}_0$, the data owner does the following:
  a) Choose the random values $w, y \in Z_p$, compute $W = g^w$, $Y = g^y$, and set $MPK = (W, Y)$, $MSK = (w, y)$.
  b) Pick a random $t_0 \in Z_p$ corresponding to initial time period $\mathcal{T}_0$, calculate $T_0 = g^{t_0}$, and set $PK_0 = T_0$, $SK_0 = t_0$.
  c) Initialize the revocation list $RL_0 = \emptyset$.
- $Extract(MSK, u_i, S_i)$: To generate the aggregate key of data class set S_i for user u_i, the data owner randomly chooses a first-order polynomial q_i such that $q_i(0) = w$ and returns $K_{S_i,u_i} = \prod_{k \in S_i} (g_{n+1-k})^{\frac{q_i(1)}{y}}$.
- $Update(\mathcal{T}_j)$: On input the current time $\mathcal{T}_j$, the data owner chooses a random $t_j \in Z_p$, computes $T_j = g^{t_j}$, and sets $PK_j = T_j$, $SK_j = t_j$.
- $Enc(MPK, PK_j, l, M)$: To encrypt a data M belonging to class l at time period $\mathcal{T}_j$, the data owner randomly chooses $z \in Z_p$, and returns $CT^j_l = (C_0, C_1, C_2, C_3, C_4)$, where

$$C_0 = M \cdot e(g_1, g_n)^z, \quad C_1 = g^z,$$
$$C_2 = Y^z, \quad C_3 = T_j^z, \quad C_4 = (W \cdot g_l)^z.$$

- $Revoke(RL_{j-1}, \mathcal{T}_j, RI_j)$: On input the revocation list RL_{j-1} at time period $\mathcal{T}_{j-1}$, the current time $\mathcal{T}_j$, and the set RI_j of user identities revoked at $\mathcal{T}_j$, the data owner updates the revocation list by $RL_j \leftarrow RL_{j-1} \cup \{RI_j\}$ and returns the updated revocation list RL_j. The data owner also sends RI_j to the server. Then, for each $u_i \in RI_j$, the cloud server delete all his updated keys set $\{ku^1_{S_i,u_i}, \ldots, ku^{j-1}_{S_i,u_i}\}$.
- $KeyUp(MSK, SK_j, RL_j, u_i, S_i)$: To generate the updated key of time period $\mathcal{T}_j$ for each user $u_i \notin RL_j$, the data owner performs the following:
  a) Recall the polynomial q_i and set S_i.
  b) Choose $x_i \in Z_p$ $(x_i \neq 0, 1)$ and compute $ku^j_{S_i,u_i} = \prod_{k \in S_i} (g_{n+1-k})^{\frac{q_i(x_i)}{t_j}}$.
  c) Set $KU_j = \{u_i, ku^j_{S_i,u_i}\}_{u_i \notin RL_j}$.
  Return $\{\mathcal{T}_j, KU_j\}$.
- $Transform(ku^j_{S_i,u_i}, CT^j_l)$: To help the user u_i $(u_i \notin RL_j)$ transform a ciphertext CT^j_l $(l \in S_i)$ into a partially decrypted ciphertext, the server searches $(u_i, ku^j_{S_i,u_i})$ from the set KU_j, computes $CT^{j'}_l =$

$$C_0 \frac{e(ku^j_{S_i,u_i}, C_3)^{\triangle_{x_i,C}(0)} \cdot e(\prod_{k \in S_i, k \neq l} g_{n+1-k+l}, C_1)}{e(\prod_{k \in S_i} g_{n+1-k}, C_4)},$$

  where $C = \{1, x_i\}$, and returns $CT^j_{l,u_i} = \{CT^{j'}_l, C_2\}$.
- $Dec(K_{S_i,u_i}, CT^j_{l,u_i})$: In order to recover the data M from the partially decrypted ciphertext CT^j_{l,u_i}, the user u_i with aggregate key K_{S_i,u_i} computes

$$M = e(K_{S_i,u_i}, C_2)^{\triangle_{1,C}(0)} \cdot CT^{j'}_l.$$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

Correctness: The correctness of our KAC-UR scheme is given in (1) shown at the bottom of this page.

C. Security Proof

Theorem 1: The KAC-UR scheme is IND-CPA secure assuming the decisional n-BDHE assumption holds in $\mathcal{G}$.

Proof: Assuming that there exists an adversary $\mathcal{A}$ can break the KAC-UR scheme with $Adv_{KAC-UR}^{IND-CPA}(\lambda) \geq \varepsilon$, we construct a simulator $\mathcal{B}$ that can solve the decisional n-BDHE problem at least with probability ε. Therefore, if the decisional n-BDHE assumption holds in $\mathcal{G}$, then ε must be negligible.

Given a random n-BDHE problem instance $(g, h, y_{g,\alpha,n}, Z)$ over the bilinear map group $\mathcal{PG}$, where Z is either $e(g_{n+1}, h)$ or a random element of $\mathcal{G}_1$, $\mathcal{B}$ simulates the challenger $\mathcal{C}$ and interacts with $\mathcal{A}$ to run each phase of IND-CPA Game.

- *Init:* $\mathcal{A}$ declares the data class identifier $l^* \in [1, n]$ and the time period $\mathcal{T}_*$ that it wants to attack.
- *Sys:* $\mathcal{B}$ gives $\mathcal{A}$ the public parameter

$$param =< \mathcal{PG}, g, g_1, g_2, \ldots, g_n, g_{n+2}, \ldots, g_{2n} >,$$

  which is available from the problem instance.
- *Setup:* $\mathcal{B}$ first chooses $w', y, t'_0 \in Z_p$ at random. Then, it computes $W = g^{w'} \cdot (g_{l^*})^{-1}$, $Y = g^y$, $T_0 = (g_{l^*})^{t'_0}$ (if $\mathcal{T}_* = \mathcal{T}_0$, then let $T_0 = g^{t'_0}$), and initializes an empty revocation list RL_0. Finally, it sends $MPK = (W, Y)$, $PK_0 = T_0$, and RL_0 to $\mathcal{A}$.
- *Query Phase 1:* $\mathcal{A}$ adaptively issues the following queries polynomial times.
  - When $\mathcal{A}$ makes an aggregate key query for (u_i, S_i), $\mathcal{B}$ chooses $a'_i \in Z_p$ randomly, and calculates K_{S_i,u_i} as

$$K_{S_i,u_i} = \prod_{k \in S_i} (g_{n+1-k})^{\frac{a'_i + w'}{y}}.$$

    By the definition of $MPK = (W, Y)$, we know $MSK = (w, y) = (w' - \alpha^{l^*}, y)$. And we let $q_i(x) = a_i x + w = (a'_i + \alpha^{l^*})x + (w' - \alpha^{l^*})$, then we have

$$\begin{aligned} K_{S_i,u_i} &= \prod_{k \in S_i} (g_{n+1-k})^{\frac{q_i(1)}{y}} \\ &= \prod_{k \in S_i} (g_{n+1-k})^{\frac{a'_i + \alpha^{l^*} + w' - \alpha^{l^*}}{y}} \\ &= \prod_{k \in S_i} (g_{n+1-k})^{\frac{a'_i + w'}{y}} \end{aligned}$$

    as required. We notice that $k \neq 0$, so the product doesn't contain g_{n+1}. Consequently, $\mathcal{B}$ holds all of the values required to compute K_{S_i,u_i}.
  - When $\mathcal{A}$ makes an update query for $\mathcal{T}_j$, $\mathcal{B}$ randomly picks $t'_j \in Z_p$, computes

$$T_j = \begin{cases} (g_{l^*})^{t'_j} & \text{if } \mathcal{T}_j \neq \mathcal{T}_*, \\ g^{t'_*} & \text{if } \mathcal{T}_j = \mathcal{T}_*, \end{cases}$$

    and finally returns $PK_j = T_j$ to $\mathcal{A}$.
  - When $\mathcal{A}$ makes a revocation query for $(\mathcal{T}_j, RI_j)$, $\mathcal{B}$ first recalls the revocation list RL_{j-1} of time period $\mathcal{T}_{j-1}$, then updates the revocation list by $RL_j \leftarrow RL_{j-1} \cup \{RI_j\}$, and finally returns RL_j to $\mathcal{A}$. The restriction is that if $\mathcal{A}$ requests an aggregate key for (u_i, S_i) satisfying $l^* \in S_i$, then the revocation query must be queried on $(\mathcal{T}_j, RI_j)$ with $u_i \in RI_j$ for any $\mathcal{T}_j \leq \mathcal{T}_*$.
  - When $\mathcal{A}$ makes a key update query for $(u_i, S_i, \mathcal{T}_j)$, $\mathcal{B}$ responds according to the value of $\mathcal{T}_j$. Specifically,
    If $\mathcal{T}_j \neq \mathcal{T}_*$, $\mathcal{B}$ recalls a'_i and t'_j, and computes

$$ku^j_{S_i,u_i} = \left(\prod_{k \in S_i} g_{n+1-k} \right)^{-\frac{w' + a'_i}{a'_i t'_j}}.$$

    Indeed, we know $SK_j = t_j = \alpha^{l^*} t'_j$ (by the definition of PK_j) and $q_i(x) = (a'_i + \alpha^{l^*})x + (w' - \alpha^{l^*})$. We let

$$\begin{aligned} & e(K_{S_i,u_i}, C_2)^{\triangle_{1,C}(0)} \cdot CT_l^{j'} \\ &= C_0 \frac{e(K_{S_i,u_i}, C_2)^{\triangle_{1,C}(0)} \cdot e(ku^j_{S_i,u_i}, C_3)^{\triangle_{x_i,C}(0)} \cdot e(\prod_{k \in S_i, k \neq l} g_{n+1-k+l}, C_1)}{e(\prod_{k \in S_i} g_{n+1-k}, C_4)} \\ &= C_0 \frac{e(\prod_{k \in S_i} (g_{n+1-k})^{\frac{q_i(1)}{y}}, g^{yz})^{\triangle_{1,C}(0)} \cdot e(\prod_{k \in S_i} (g_{n+1-k})^{\frac{q_i(x_i)}{t_j}}, g^{t_j z})^{\triangle_{x_i,C}(0)} \cdot e(\prod_{k \in S_i, k \neq l} g_{n+1-k+l}, g^z)}{e(\prod_{k \in S_i} g_{n+1-k}, (g^w \cdot g_l)^z)} \\ &= C_0 \frac{e(\prod_{k \in S_i} g_{n+1-k}, g)^{z(q_i(1)\triangle_{1,C}(0) + q_i(x_i)\triangle_{x_i,C}(0))} \cdot e(\prod_{k \in S_i, k \neq l} g_{n+1-k+l}, g^z)}{e(\prod_{k \in S_i} g_{n+1-k}, g^{wz}) \cdot e(\prod_{k \in S_i} g_{n+1-k}, g_l^z)} \\ &= C_0 \frac{e(\prod_{k \in S_i} g_{n+1-k}, g)^{zw} \cdot e(\prod_{k \in S_i, k \neq l} g_{n+1-k+l}, g^z)}{e(\prod_{k \in S_i} g_{n+1-k}, g^{wz}) \cdot e(\prod_{k \in S_i} g_{n+1-k}, g_l^z)} \\ &= M \cdot e(g_1, g_n)^z \frac{1}{e(g_{n+1}, g)^z} \\ &= M \end{aligned} \tag{1}$$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

$x_i = -\frac{w'}{a'_i}$, then we have

$$\begin{aligned} ku^j_{S_i,u_i} &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{q_i(x_i)}{t_j}} \\ &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{(a'_i+\alpha^{l^*})(-\frac{w'}{a'_i})+(w'-\alpha^{l^*})}{\alpha^{l^*}t'_j}} \\ &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{-w'-\frac{\alpha^{l^*}w'}{a'_i}+w'-\alpha^{l^*}}{\alpha^{l^*}t'_j}} \\ &= \prod_{k\in S_i} (g_{n+1-k})^{-\frac{w'+a'_i}{a'_i t'_j}} \end{aligned}$$

as required. Obviously, $\mathcal{B}$ has all necessary values to compute $ku^j_{S_i,u_i}$.

If $\mathcal{T}_j = \mathcal{T}_*$, then $l^* \notin S_i$. $\mathcal{B}$ recalls a'_i and t'_*, randomly picks $x_i \in Z_p$, and computes

$$ku^*_{S_i,u_i} = \prod_{k\in S_i} (g_{n+1-k})^{\frac{a'_i x_i + w'}{t'_*}} \cdot (g_{n+1-k+l^*})^{\frac{x_i-1}{t'_*}}.$$

Indeed, we know $SK_* = t'_*$ (by the definition of PK_*) and $q_i(x) = (a'_i + \alpha^{l^*})x + (w' - \alpha^{l^*})$, then we have

$$\begin{aligned} ku^*_{S_i,u_i} &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{q_i(x_i)}{t'_*}} \\ &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{(a'_i+\alpha^{l^*})x_i+(w'-\alpha^{l^*})}{t'_*}} \\ &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{a'_i x_i+w'}{t'_*}} \cdot (g_{n+1-k})^{\frac{(x_i-1)\alpha^{l^*}}{t'_*}} \\ &= \prod_{k\in S_i} (g_{n+1-k})^{\frac{a'_i x_i+w'}{t'_*}} \cdot (g_{n+1-k+l^*})^{\frac{x_i-1}{t'_*}} \end{aligned}$$

as required. The main point is that $l^* \notin S_i$ and hence $\prod_{k\in S_i} g_{n+1-k+l^*}$ does not include the term g_{n+1}. Therefore, $\mathcal{B}$ has all necessary values to compute $ku^*_{S_i,u_i}$.

- *Challenge:* $\mathcal{A}$ sends $\mathcal{B}$ two messages of the same length, M_0 and M_1, belonging to class l^*. $\mathcal{B}$ randomly selects a bit b and calculates the challenge ciphertext

$$\begin{aligned} CT^* &= (C^*_0, C^*_1, C^*_2, C^*_3, C^*_4) \\ &= (M_b \cdot Z, h, h^y, h^{t'_*}, h^{w'}) \end{aligned}$$

Let $z = log_g h$, then when $Z = e(g_{n+1}, h)$,

$$\begin{aligned} CT^* &= (M_b \cdot Z, h, h^y, h^{t'_*}, h^{w'}) \\ &= (M_b \cdot e(g_{n+1}, g^z), g^z, g^{zy}, g^{zt'_*}, g^{zw'}) \\ &= (M_b \cdot e(g_1, g_n)^z, g^z, (g^y)^z, (g^{t'_*})^z, (g^w \cdot g^{\alpha^{l^*}})^z) \\ &= (M_b \cdot (g_1, g_n)^z, g^z, Y^z, T^z_*, (W \cdot g_{l^*})^z) \end{aligned}$$

Accordingly, by definition, CT^* is a valid challenge ciphertext for M_b.

- *Query Pahse 2:* $\mathcal{A}$ continues the querying as in Query Phase 1.
- *Guess:* $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ for b. If $b' = b$, $\mathcal{B}$ outputs 1 meaning $Z = e(g_{n+1}, h)$. Otherwise, it outputs 0 meaning Z is random in $\mathcal{G}_1$.

This completes the simulation and solution. We then analyze the probability that $\mathcal{B}$ correctly solves the decisional n-BDHE problem with ε.

If the problem instance is sampled from T_{BDHE}, simulation and real attack are indistinguishable, and therefore the probability that $\mathcal{A}$ outputs $b' = b$ is at least $\frac{1}{2} + \varepsilon$. If the problem instance is sampled from F_{BDHE}, $\mathcal{A}$ guesses $b' = b$ with probability $\frac{1}{2}$. Therefore, we have

$$\begin{aligned} &|Pr[\mathcal{B}(g, h, y_{g,\alpha,n}, e(g_{n+1}, h)) = 1] \\ &- Pr[\mathcal{B}(g, h, y_{g,\alpha,n}, Z) = 1]| \\ \geq &\left(\frac{1}{2} + \varepsilon\right) - \frac{1}{2} \\ = &\ \varepsilon \end{aligned}$$

The proof is concluded. □

TABLE III
MULTILINEAR MAP RUNNING COST

n	2	3	5	7
Time (s)	898	3639	24554	27295

V. PERFORMANCE

A. Theoretical Analysis

In this section, we analyze and compare the efficiency of the KAC-UR scheme and the scheme proposed in [11] theoretically and experimentally. Although [12] also proposed a revocable KAC scheme, this scheme is based on the multilinear map which is very computationally expensive. Specifically, we test the performance of the multilinear map open-source library. Our experiments are based on GMP6.2.1 large integer computing library, running on AMD Ryzen 5 4600H CPU 3.00 GHz and 3 GB memory environment. We denote a multilinear map as $e : \mathcal{G}_1 \times \mathcal{G}_2 \times \cdots \times \mathcal{G}_n \to \mathcal{G}_T$, and choose the security parameter $\lambda = 80$. We separately test the time cost of generating a multilinear map group system when $n = 2, 3, 5, 7$, as shown in Table III. It can be seen that when $n = 7$, the time to generate a multilinear map group system will exceed 7.5 hours. In the scheme of [12], n is the number of data classes. In practical applications, the scale of data in cloud storage is usually massive. Therefore, the cost required to generate a multilinear map group system is beyond the user's tolerance, and it is even more unrealistic to use this solution for data sharing. Therefore, we will not analyze the performance of [12] in detail below.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPUTATION COMPARISON WITH [11]

Schemes		Patranabis et al. [11]	Our scheme
Data Owner	Sys	$(2n-1)E_{\mathcal{G}}+(2n-2)M_{Z_p}$	$(2n-1)E_{\mathcal{G}}+(2n-2)M_{Z_p}$
	Setup	$E_{\mathcal{G}}$	$3E_{\mathcal{G}}$
	Extract	$E_{\mathcal{G}}+(\|S\|-1)M_{\mathcal{G}}$	$E_{\mathcal{G}}+(\|S\|-1)M_{\mathcal{G}}+M_{Z_p}$
	Encryption	$P+2E_{\mathcal{G}}+E_{\mathcal{G}_1}+M_{\mathcal{G}}+M_{\mathcal{G}_1}$	$P+4E_{\mathcal{G}}+E_{\mathcal{G}_1}+M_{\mathcal{G}}+M_{\mathcal{G}_1}$
	Key Update	$E_{\mathcal{G}}$	$\|URL_{\mathcal{T}}\|(E_{\mathcal{G}}+(\|S\|-1)M_{\mathcal{G}}+M_{Z_p})$
	Ciphertext Re-encryption	$n(2P+3E_{\mathcal{G}}+2E_{\mathcal{G}_1}+E_{Z_p}+M_{\mathcal{G}}+2M_{\mathcal{G}_1})$	$\perp$
Cloud Server	Decryption	$\perp$	$3P+E_{\mathcal{G}_1}+(2\|S\|-3)M_{\mathcal{G}}+3M_{\mathcal{G}_1}$
Data User	Decryption	$2P+(2\|S\|-1)M_{\mathcal{G}}+2M_{\mathcal{G}_1}$	$P+E_{\mathcal{G}_1}+M_{\mathcal{G}_1}$

TABLE V
STORAGE COMPARISON WITH [11]

Schemes		Patranabis et al. [11]	Our scheme
Data Owner	Master Secret Key	$\|Z_p\|$	$2\|Z_p\|$
	Secret Key	$2\|Z_p\|$	$\|Z_p\|$
Cloud Server	Ciphertext	$n(2\|\mathcal{G}\|+\|\mathcal{G}_1\|)$	$n(4\|\mathcal{G}\|+\|\mathcal{G}_1\|)$
	Updated Key	$\perp$	$\|URL_{\mathcal{T}}\|\ \|\mathcal{T}\|\ \|\mathcal{G}\|$
Data User	Aggregate Key	$\|\mathcal{G}\|$	$\|\mathcal{G}\|$
	Updated Key	$\|\mathcal{G}\|$	$\perp$

TABLE VI
COMMUNICATION COMPARISON WITH [11]

Schemes	Patranabis et al. [11]			Our scheme
Ciphertext Outsourcing	DO $\rightarrow$ CS $n(2\|\mathcal{G}\|+\|\mathcal{G}_1\|)$			DO $\rightarrow$ CS $n(4\|\mathcal{G}\|+\|\mathcal{G}_1\|)$
Authorization	DO $\rightarrow$ DU $m\|\mathcal{G}\|$			DO $\rightarrow$ DU $m\|\mathcal{G}\|$
Revocation	DO $\rightarrow$ DU $\|URL_{\mathcal{T}}\|\ \|\mathcal{G}\|$	CS $\rightarrow$ DO $n(2\|\mathcal{G}\|+\|\mathcal{G}_1\|)$	DO $\rightarrow$ CS $n(2\|\mathcal{G}\|+\|\mathcal{G}_1\|)$	DO $\rightarrow$ CS $\|URL_{\mathcal{T}}\|\ \|\mathcal{G}\|$
Ciphertext Access	CS $\rightarrow$ DU $\|S\|(2\|\mathcal{G}\|+\|\mathcal{G}_1\|)$			CS $\rightarrow$ DU $\|S\|(\mathcal{G}+\mathcal{G}_1)$

We compare our scheme with that of Patranabis et al. [11] in computation, storage, and communication. Table IV is the computation comparison, Table V is the storage comparison, and Table VI is the communication comparison.

For ease of comparison, we regard the access control parameter in [11] as the updated key. As in our scheme, when a new time period is up, the data owner generates updated keys (access control parameter) for all unrevoked users. A major difference, as well as our main contribution, is that under our scheme, each user's updated key is different. The aggregate key and updated key of different users cannot be combined, so that our scheme is resistant to collusion attacks. Consequently, updated keys can be stored on the cloud server, which effectively saves the user's communication and storage overhead. However, all users in [11] have the same updated key, and the revoked user can easily collude with any unrevoked user.

We define the notations used in the Tables. Let P, $E_{\mathcal{G}}$, $E_{\mathcal{G}_1}$, E_{Z_p}, $M_{\mathcal{G}}$, $M_{\mathcal{G}_1}$, and M_{Z_p} denote the computation cost of a bilinear pairing, an exponentiation in $\mathcal{G}$, an exponentiation in $\mathcal{G}_1$, an exponentiation in Z_p, a multiplication in $\mathcal{G}$, a multiplication in $\mathcal{G}_1$, and a multiplication in Z_p, respectively. Let $|Z_p|$, $|\mathcal{G}|$, and $|\mathcal{G}_1|$ denote the bit-length of an element in groups Z_p, $\mathcal{G}$, and $\mathcal{G}_1$, respectively. Let m denote the total number of users in the initial authorization phase, $\mathcal{T}$ denote the current time period, and $|\mathcal{T}|$ denote the total number of time periods experienced so far. $RL_{\mathcal{T}}$ is the revocation list at time period $\mathcal{T}$. Let $|URL_{\mathcal{T}}|$ denote the number of unrevoked users until time period $\mathcal{T}$. For convenience, we assume that each data class contains only one data and the number of data classes that the data owner authorizes each user to access is $|S|$.

Table IV shows that the difference in computational overhead between our scheme and [11] is mainly in the key update, ciphertext re-encryption, and decryption. Although our scheme requires more computational overhead when updating the key, it is to generate different updated keys for each user to resist collusion attacks. To prevent the revoked users from decrypting ciphertexts encrypted before being revoked, our scheme only requires the cloud server to delete the updated keys of all revoked users. However, [11] requires the data owner to re-encrypt all ciphertexts stored in the cloud server at each revocation, which is very computationally expensive. With the update of the time period and the increase in the number of ciphertexts, this will become a bottleneck for a data owner with limited computing resources. In addition, [11] has a linear overhead in user decryption, i.e., $\mathcal{O}|S|$, while our scheme only requires constant cost, which is due to the fact that we delegate most of the computation to the powerful cloud server.

Table V shows that our scheme offloads most of the key storage overhead to the cloud server, leaving only one group element for the user to store.

Table VI shows the communication comparison between our scheme and [11] in the four phases of ciphertext outsourcing, authorization, revocation, and ciphertext access. It can be seen that the main difference between the two schemes is the revocation phase. Our scheme does not require the secure channel between the user and the data owner during key updating. But [11] requires that the unrevoked user interacts with the data owner over

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

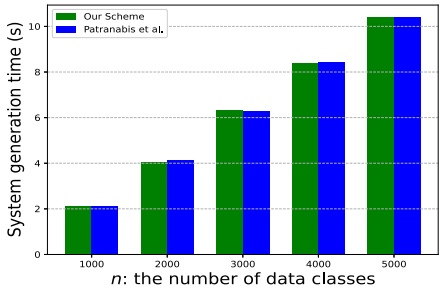

Fig. 4. System parameter generation time.

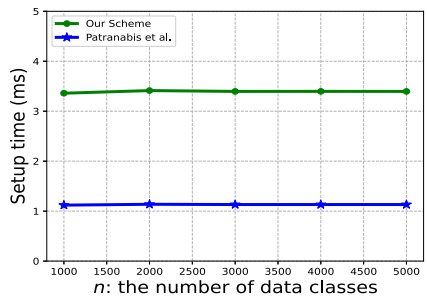

Fig. 5. Setup time.

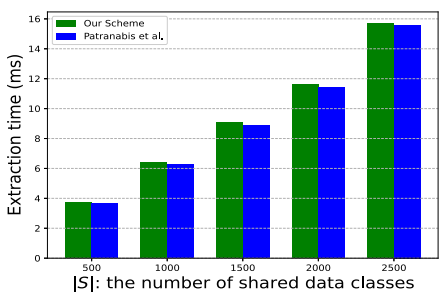

Fig. 6. Extraction time.

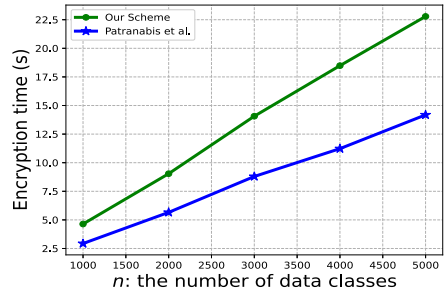

Fig. 7. Encryption time.

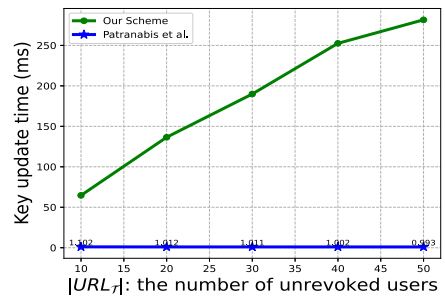

Fig. 8. Key update time.

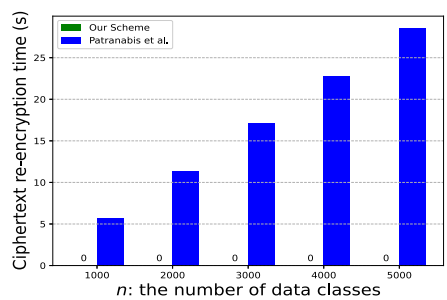

Fig. 9. Ciphertext re-encryption time.

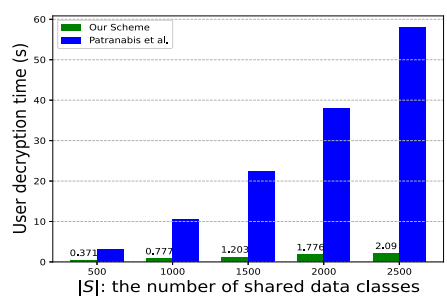

Fig. 10. User decryption time.

a secure channel to obtain the updated key for each revocation. In addition, to perform ciphertext re-encryption, [11] needs the data owner to download all ciphertexts from the cloud server and upload the re-encrypted ciphertexts to the cloud server. This communication overhead and data maintenance overhead are huge, especially for large databases.

B. Practical Analysis

In this subsection, we provide a series of experimental evaluations of our KAC-UR scheme. Our experiments are performed on Ubuntu 20.04.1 LTS operating system and are implemented by GCC 9.4.0 compile on a Linux Server with AMD Ryzen 5 4600H with Radeon Graphics CPU@ 3.00 GHz and 3 GB memory by using C++ programming language based on the Paring Based Cryptography (PBC) Library. We benchmark the computation and communication costs of [11] and our scheme in different settings.

Figs. 4, 5, 6, 7, 8, 9, and 10 show the computation costs.

Fig. 4 shows the cost of system parameter generation. It indicates that the two schemes require the same overhead to generate system parameters, which is consistent with theoretical analysis. The time cost of the Setup algorithm is shown in Fig. 5. Although the overhead of our scheme is larger, it is still on the millisecond level and remains constant size. Fig. 6 shows the running time of the Extract algorithm, indicating that the

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

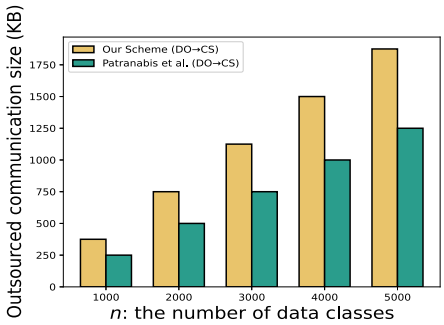

Fig. 11. Ciphertext outsourcing communication size.

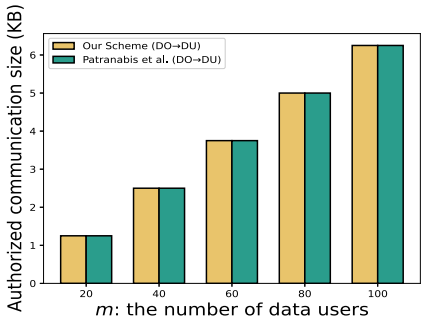

Fig. 12. Authorization communication size.

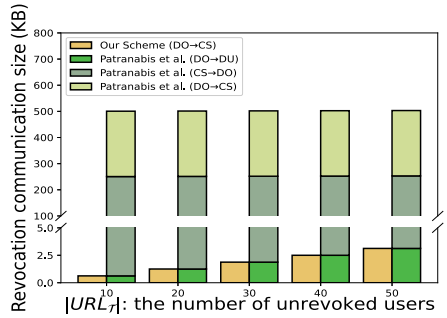

Fig. 13. Revocation communication size.

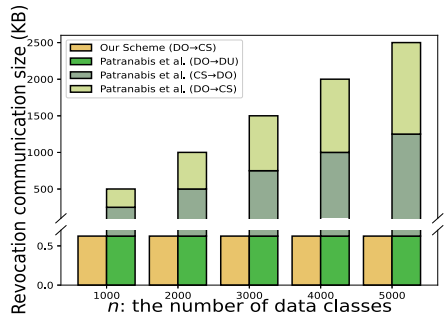

Fig. 14. Revocation communication size.

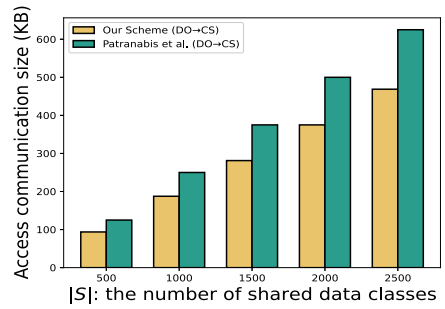

Fig. 15. Ciphertext access communication size.

efficiency of the extract algorithm of these two schemes is almost the same, and has almost a linear relationship with the amount of data classes, which matches theoretical analysis.

The time cost of the Enc algorithm and KeyUp algorithm are shown in Figs. 7 and 8 respectively, where in Fig. 8 we fix $|S| = 1000$. The simulation results of Figs. 7 and 8 indicate that our scheme requires more overhead than the scheme in [11]. This is because we embed the time component into ciphertext and distinct time components into each user's updated key to resist collusion attack, resulting in some additional computations needed in our scheme.

Fig. 9 shows the time cost of ciphertext re-encryption. It reveals that the time consumed in Patranabis et al.'s scheme is linear with the number of data classes n, while in our scheme the ciphertext re-encryption is not needed. Combining Figs. 8 and 9, it can be seen that in each revocation phase, although our scheme has a larger key update overhead than [11], the key update time of our scheme is much smaller than the ciphertext re-encryption time of [11]. Therefore, the revocation efficiency of our scheme is significantly higher than [11].

In Fig. 10, we give the user-side decryption efficiency comparison of the two schemes. It shows that our scheme runs much less time than that of [11] for the same value $|S|$, because we outsource most of the overhead of decryption to the powerful cloud server.

Figs. 11, 12, 13, 14, and 15 show the communication costs.

Fig. 11 shows the communication size in the ciphertext outsourcing phase. In this phase, the communication overhead of both schemes is linearly related to the number of data classes. The size of our ciphertext communication is slightly larger than [11]. This is because our scheme splits the aggregate key into two parts to achieve user revocation, so the corresponding ciphertext must also introduce two elements, but our scheme achieves collusion resistance.

The communication size of the authorization phase is shown in Fig. 12. It can be seen that the communication size of the two schemes in this phase is the same and has a linear relationship with the number of users, which is consistent with the theoretical analysis.

Figs. 13 and 14 show the revocation communication size, indicating that the revocation communication of our scheme is much smaller than [11]. In Fig. 13, we fix $n = 1000$ and test the trend of revocation communication overhead relative to the number of unrevoked users $|URL_{\mathcal{T}}|$. The simulation result shows that the communication size between DO and CS in our scheme in this phase is the same as that between DO and DU in [11] and is linearly related to $|URL_{\mathcal{T}}|$, but [11] also requires ciphertext

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

re-encryption, which brings huge communication overhead to the system. In Fig. 14, we fix $|URL_{\mathcal{T}}| = 10$ and test the trend of revocation communication overhead relative to the number of data classes. The simulation result indicates that the communication overhead of our scheme in this phase is independent of n, and when $|URL_{\mathcal{T}}| = 10$, the communication overhead is only about 0.6 KB. However, the ciphertext re-encryption operation in [11] brings huge overheads to the revocation phase and is related to n.

Fig. 15 gives the communication size of the ciphertext access phase. In our scheme, the cloud server performs partial decryption operations for the user, so there is no need to return the original ciphertext to the user, but only the partially decrypted ciphertext. Furthermore, the partially decrypted ciphertext size of our scheme is smaller than the original ciphertext size of [11], therefore, the communication overhead of our scheme is smaller in this phase.

VI. Conclusion

This paper investigates the crucial necessity for revocation in key-aggregate cryptosystems and proposes a key-aggregate cryptosystem with user revocation (KAC-UR) scheme. Our scheme allows data owners to share encrypted data flexibly. More importantly, it achieves secure and efficient user revocation. Compared with previous solutions, our scheme significantly reduces overhead for data owners and users caused by revocation. Our scheme also enables the cloud server to perform partial decryption, markedly decreasing the user's decryption overhead. Rigorous proof shows that the proposed scheme is IND-CPA secure. Theoretical analysis and experimental results show the proposed scheme's practicality.

References

[1] M. J. Haber, B. Chappell, and C. Hills, "Cloud computing," in *Cloud Attack Vectors*. Berlin, Germany: Springer, 2022, pp. 9–25.

[2] Q. Tong, Y. Miao, J. Weng, X. Liu, K.-K. R. Choo, and R. H. Deng, "Verifiable fuzzy multi-keyword search over encrypted data with adaptive security," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 5386–5399, May 2023.

[3] W. Shen, J. Yu, M. Yang, and J. Hu, "Efficient identity-based data integrity auditing with key-exposure resistance for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4593–4606, Nov./Dec. 2023.

[4] K. Ren, C. Wang, and Q. Wang, "Security challenges for the public cloud," *IEEE Internet Comput.*, vol. 16, no. 1, pp. 69–73, Jan./Feb. 2012.

[5] K. Liang, C. Su, J. Chen, and J. K. Liu, "Efficient multi-function data sharing and searching mechanism for cloud-based encrypted data," in *Proc. 11th ACM Asia Conf. Comput. Commun. Secur.*, 2016, pp. 83–94.

[6] F. Song, Z. Qin, J. Zhang, D. Liu, J. Liang, and X. S. Shen, "Efficient and privacy-preserving outsourced image retrieval in public clouds," in *Proc. IEEE Glob. Commun. Conf.*, 2020, pp. 1–6.

[7] J. Liang, Z. Qin, S. Xiao, L. Ou, and X. Lin, "Efficient and secure decision tree classification for cloud-assisted online diagnosis services," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 4, pp. 1632–1644, Jul./Aug. 2021.

[8] J. Liang, Z. Qin, S. Xiao, J. Zhang, H. Yin, and K. Li, "Privacy-preserving range query over multi-source electronic health records in public clouds," *J. Parallel Distrib. Comput.*, vol. 135, pp. 127–139, 2020.

[9] H. Yang, Y. Su, J. Qin, and H. Wang, "Privacy-preserving outsourced inner product computation on encrypted database," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1320–1337, Mar./Apr. 2022.

[10] C.-K. Chu, S. S. Chow, W.-G. Tzeng, J. Zhou, and R. H. Deng, "Key-aggregate cryptosystem for scalable data sharing in cloud storage," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 2, pp. 468–477, Feb. 2014.

[11] S. Patranabis, Y. Shrivastava, and D. Mukhopadhyay, "Dynamic key-aggregate cryptosystem on elliptic curves for online data sharing," in *Proc. Int. Conf. Cryptol. India*, Springer, 2015, pp. 25–44.

[12] Q. Gan, X. Wang, and D. Wu, "Revocable key-aggregate cryptosystem for data sharing in cloud," *Secur. Commun. Netw.*, vol. 2017, pp. 1–11, 2017.

[13] D. Boneh and A. Silverberg, "Applications of multilinear forms to cryptography," *Contemporary Math.*, vol. 324, no. 1, pp. 71–90, 2003.

[14] D. Naor, M. Naor, and J. Lotspiech, "Revocation and tracing schemes for stateless receivers," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 2001, pp. 41–62.

[15] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[16] D. Thilakanathan, S. Chen, S. Nepal, and R. A. Calvo, "Secure data sharing in the cloud," in *Security, Privacy and Trust in Cloud Systems*. Berlin, Germany: Springer, 2014, pp. 45–72.

[17] J. Liu, B. Zhao, J. Qin, X. Zhang, and J. Ma, "Multi-keyword ranked searchable encryption with the wildcard keyword for data sharing in cloud computing," *Comput. J.*, vol. 66, no. 1, pp. 184–196, 2023.

[18] X.-J. Lin, L. Sun, and H. Qu, "Cryptanalysis of an anonymous and traceable group data sharing in cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2773–2775, 2021.

[19] J. Shen, H. Yang, P. Vijayakumar, and N. Kumar, "A privacy-preserving and untraceable group data sharing scheme in cloud computing," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2198–2210, Jul./Aug. 2022.

[20] C. Lai, Z. Ma, R. Guo, and D. Zheng, "Secure medical data sharing scheme based on traceable ring signature and blockchain," *Peer-to-Peer Netw. Appl.*, vol. 15, no. 3, pp. 1562–1576, 2022.

[21] D. Boneh, C. Gentry, and B. Waters, "Collusion resistant broadcast encryption with short ciphertexts and private keys," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 2005, pp. 258–275.

[22] S. Patranabis, Y. Shrivastava, and D. Mukhopadhyay, "Provably secure key-aggregate cryptosystems with broadcast aggregate keys for online data sharing on the cloud," *IEEE Trans. Comput.*, vol. 66, no. 5, pp. 891–904, May 2017.

[23] Z. Wang, "Provably secure key-aggregate cryptosystems with auxiliary inputs for data sharing on the cloud," *Future Gener. Comput. Syst.*, vol. 93, pp. 770–776, 2019.

[24] C. Guo et al., "Key-aggregate authentication cryptosystem for data sharing in dynamic cloud storage," *Future Gener. Comput. Syst.*, vol. 84, pp. 190–199, 2018.

[25] K. Alimohammadi, M. Bayat, and H. H. Javadi, "A secure key-aggregate authentication cryptosystem for data sharing in dynamic cloud storage," *Multimedia Tools Appl.*, vol. 79, no. 3, pp. 2855–2872, 2020.

[26] J. Liu, J. Qin, W. Wang, L. Mei, and H. Wang, "Key-aggregate based access control encryption for flexible cloud data sharing," *Comput. Standards Interfaces*, vol. 88, 2024, Art. no. 103800.

[27] G. Pareek and B. Purushothama, "KAPRE: Key-aggregate proxy re-encryption for secure and flexible data sharing in cloud storage," *J. Inf. Secur. Appl.*, vol. 63, 2021, Art. no. 103009.

[28] S. Patranabis and D. Mukhopadhyay, "Identity-based key aggregate cryptosystem from multilinear maps," *Adv. Math. Commun.*, vol. 13, no. 4, pp. 759–778, 2019.

[29] C.-C. Lee, C.-T. Li, S.-T. Chiu, and S.-D. Chen, "Time-bound key-aggregate encryption for cloud storage," *Secur. Commun. Netw.*, vol. 9, no. 13, pp. 2059–2069, 2016.

[30] B. Cui, Z. Liu, and L. Wang, "Key-aggregate searchable encryption (KASE) for group data sharing via cloud storage," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2374–2385, Aug. 2016.

[31] Z. Liu, T. Li, P. Li, C. Jia, and J. Li, "Verifiable searchable encryption with aggregate keys for data sharing system," *Future Gener. Comput. Syst.*, vol. 78, pp. 778–788, 2018.

[32] J. Liu, B. Zhao, J. Qin, X. Hou, and J. Ma, "Key-aggregate searchable encryption supporting conjunctive queries for flexible data sharing in the cloud," *Inf. Sci.*, vol. 645, 2023, Art. no. 119336.

[33] J. Liu, Z. Wei, J. Qin, B. Zhao, and J. Ma, "Verifiable key-aggregate searchable encryption with a designated server in multi-owner setting," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4233–4247, Nov./Dec. 2023.

[34] D. Boneh and M. Franklin, "Identity-based encryption from the weil pairing," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 2001, pp. 213–229.

[35] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 2005, pp. 457–473.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.

Jinlu Liu received the BS degree from the School of Mathematical Sciences, Shanxi University, P. R. China, in 2019. She is currently working toward the PhD degree with the School of Mathematics, Shandong University. She was a visiting PhD student with the School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore. Her research interests include cloud computing and applied cryptography.

Jing Qin received the BS degree from Information Engineering University, Zhenzhou, P. R. China, in 1982, and the PhD degree from the School of Mathematics, Shandong University, P. R. China, in 2004. She is a professor with the School of Mathematics, Shandong University P. R. China. Her research interests include information security, design and analysis of security about cryptologic protocols. She has coauthored two books and has published more than 30 professional research papers. She is a senior member of the Chinese Association for Cryptologic Research (CACR) as well as China Computer Federation (CCF).

Xi Zhang received the BS degree from the School of Mathematical Sciences, Hebei Normal University, P. R. China, in 2018. She is currently working toward the PhD degree with the School of Mathematics, Shandong University. Her research interests include searchable encryption and cloud computing.

Huaxiong Wang received the PhD degree in mathematics from the University of Haifa, Israel, in 1996, and the PhD degree in computer science from the University of Wollongong, Australia, in 2001. He joined Nanyang Technological University in 2006 and is currently a professor with the Division of Mathematical Sciences. He is also an honorary fellow with Macquarie University, Australia. His research interests include cryptography, information security, coding theory, combinatorics, and theoretical computer science. He has served on the editorial boards of three international journals: the *Design, Coding and Cryptography* (2006-2011), *Journal of Communications* (JCM), and *Journal of Communications and Networking*. He was a program co-chair of the 9th Australian Conference on Information Security and Privacy (ACISP 04) in 2004 and the 4th International Conference on Cryptography and Cybersecurity (CANS 05) in 2005, and has served on program committees at more than 70 international conferences. In 2004, he received the inaugural Best Contribution to Research Award from the Australian Computer Science Society.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on October 14,2025 at 04:45:53 UTC from IEEE Xplore. Restrictions apply.